

ROS 2 Autonomous Sphere Collector Robot

Project Submission Links

Item	Status	Link
GitHub Repository	Required	[https://github.com/ArmaanMeh/autonomous_sphere_collector.git]
Video Demonstration	Required	[https://youtu.be/J2sMdY9UO-w]

Introduction & Objective

This project documents the development and current state of a ROS 2-based mobile robot system designed for the autonomous task of locating, collecting, and delivering three spheres of varying size and color to a designated goal zone within a mapped environment.

The robot platform features a custom model, `ros2_control` hardware abstraction, and was originally intended to use the Nav2 stack for autonomous movement.

System Goal (Original Objective)

The original objective was to autonomously complete the following sequence for all three spheres:

1. Detect the position and estimated radius of a sphere via **LiDAR-only geometric clustering**.
2. Navigate to an approach pose.
3. Collect the sphere using the bulldozer bucket.
4. Navigate to the predefined goal zone and deliver the ball.

Setup and Installation

This project is built and tested on **ROS 2 Jazzy** and relies on **Gazebo Harmonic** (via `ros_gz_sim`).

1. Install ROS 2 Jazzy and Gazebo Harmonic

If you do not have the required ROS environment, follow the official installation guide. The system also requires the full Nav2 suite.

```
# 1. Update and setup locale
sudo apt update && sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

```
# 2. Add ROS 2 repository
sudo apt install software-properties-common
sudo add-apt-repository universe
sudo apt update && sudo apt install curl
sudo curl -sSL
[https://raw.githubusercontent.com/ros/rosdistro/master/ros.key]
(https://raw.githubusercontent.com/ros/rosdistro/master/ros.key) -o
/usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/usr/share/keyrings/ros-archive-keyring.gpg]
[http://packages.ros.org/ros2/ubuntu](http://packages.ros.org/ros2/ubuntu)
$(. /etc/os-release && echo $UBUNTU_CODENAME) main" | sudo tee
/etc/apt/sources.list.d/ros2.list > /dev/null
```

3. Install ROS 2 Jazzy (Desktop recommended)

```
sudo apt update
sudo apt install ros-jazzy-desktop
```

4. Install Gazebo Harmonic (via ros-gz)

```
sudo apt install ros-jazzy-ros-gz-sim
```

5. Install Nav2 and other dependencies (Crucial for launch files)

```
sudo apt install ros-jazzy-navigation2 ros-jazzy-nav2-bringup ros-jazzy-
slam-toolbox
```

2. Project Clone, Build, and Source Bash

Create and navigate to the workspace source directory

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
```

Clone the autonomous_sphere_collector_pkg repository

git clone https://github.com/ArmaanMeh/autonomous_sphere_collector.git
autonomous_sphere_collector_pkg

Navigate back to the root of the workspace

```
cd ~/ros2_ws
```

Build the specific package

```
colcon build --packages-select autonomous_sphere_collector_pkg
```

Source the workspace setup file (required for every new terminal session)

```
source install/setup.bash
```

Full Integrated Sim

```
ros2 launch autonomous_sphere_collector_pkg gazebo_assessment.launch.py
```

Partial Success. Robot spawns, scoop works, all other advanced functions fail.

Teleoperation Node

```
``` Functional. Launches the manual control script.
```

```
Autonomous Collector ros2 launch autonomous_sphere_collector_pkg
collect_balls.launch.py Crashed. Dependent on Nav2/Locomotion.
```

```
💡 Project Approach and Architecture
```

```
Our approach focused on creating a robust, modular pipeline to tackle the
core challenge: collecting small, movable objects using limited sensing.
```

Design Philosophy: We utilized a modular Xacro-based URDF with a stable two-wheel differential-drive base. The control system (ros2\_control) separated locomotion from manipulation.

Sensing and Detection: Since the robot primarily uses a 2D LiDAR, the detection system was designed to perform geometric clustering of scan points to distinguish the three spheres by their measured "apparent radii." The system was designed to rely heavily on this geometric signature despite the presence of a camera and IMU.

Modular Automation Logic: The autonomous system was strictly separated into two custom nodes to enhance robustness and maintainability:

target\_finder.py (Detection Node): A passive subscriber that processes LiDAR data, identifies targets, and calculates the necessary approach and pickup poses. It only publishes pose arrays.

pos\_commander.py (Action Node): An active node that subscribes to the pose arrays and uses Nav2 Action Clients to execute the navigation, collection (open/advance/close), and delivery sequence (pushing the spheres into the goal).

 System Implementation and Architecture (Current State)

1. Robot Model (URDF / Xacro)

The robot is defined using a modular Xacro-based URDF.

Locomotion: Two-wheel differential-drive configuration.

Manipulation: Bulldozer-type bucket/scoop controlled by arm joints.

Sensors: A 2D Lidar, Camera, and IMU have been used.

2. Control Integration and Movement Limitations

Hardware abstraction is handled by ros2\_control.

Controller	Function	Status (Current)
diff_drive_controller	Locomotion (Wheels)	Crashed
joint_trajectory_controller	Manipulation (Arms/Scoop)	Functional

The Linear and Angular Velocity Issue: The differential drive is designed to accept commands for Linear Velocity (cmd\_vel.linear.x) and Angular Velocity (cmd\_vel.angular.z). However, the controller responsible for translating these commands to wheel actuation is failing. This prevents the robot from executing or maintaining any commanded linear or angular velocities, making all navigational functions impossible.

Available Movements (Current State): Due to the failure of the diff\_drive\_controller, the robot is stationary. The only operational movement available is control of the scoop arms via the working joint\_trajectory\_controller.

✖ Current System State and Fatal Errors

The system is currently in a critical failure state. An underlying issue is preventing core control and navigation components from initializing, rendering almost all advanced functionality inoperable.

Crashed/Inoperable Components (The "Everything is now Crashed" State)

Component	Status	Detail
Locomotion	Crashed	The diff_drive_controller is failing to activate or communicate, meaning the robot cannot move.
SLAM / Mapping	Crashed	The SLAM Toolbox node fails to start.
Map Server	Crashed	The nav2_map_server fails to initialize the map, preventing costmap generation.
Nav2 Navigation	Crashed	The entire Nav2 stack (AMCL, planners, etc.) fails to launch due to missing critical dependencies (map, working locomotion).
Autonomous Logic	Crashed	The custom detection (target_finder.py) and collection (pos_commander.py) nodes fail without a running Nav2 stack or functional topics.

Achieved/Functional Components

Component	Status	Detail
Robot Model	Functional	The robot model successfully spawns in the Gazebo environment.
Scoop/Arm Control	Functional	The joint_trajectory_controller for the scoop arms remains functional. The arms can be commanded to open and close.

✂ Launch Commands and Troubleshooting

Launch Status

Currently, only the base spawn and scoop functionality are verifiable using the integrated launch file.

Mode	Command	Status
Full Integrated Sim	ros2 launch autonomous_sphere_collector_pkg gazebo_assessment.launch.py	Partial Success. Robot spawns, scoop works, all other advanced functions fail.
Autonomous Collector	ros2 launch autonomous_sphere_collector_pkg collect_balls.launch.py	Crashed. Dependent on Nav2/Locomotion.

Key Troubleshooting Solutions (Applied, but insufficient to fix the current crash state)

We resolved several lower-level issues, but the core locomotion/Nav2 crash remains the fatal problem:

Map Server Failure Fix:

Genuine Error: The map\_server node failed to start because the necessary map files were not being copied to the correct runtime directory (install/).

Solution: An installation rule was added to the package's CMakeLists.txt to ensure the config/map directory was installed.

Controller Tolerance Violation Fix (Scoop):

Genuine Error: The joint\_trajectory\_controller reported a Position Error: -0.104470, Position Tolerance: 0.100000, causing the scoop to enter a holding mode.

Solution: The controller's configuration file was modified to

increase the allowable position error tolerance for the scoop joint to accommodate minor simulation drift.

File: config/ros\_controllers.yaml (Excerpt)  
YAML

```
scoop_controller:
 # ... other settings
 state_tolerances:
 position: 0.15 # Increased from 0.10 to resolve the error
 # ...
```

Launch File Dependency Cleanup:

Error: Launch failed due to missing package lookups (gazebo\_models, nav2\_map\_server).

Solution: Explicit dependencies were installed, and unnecessary/outdated variable lookups were removed from the launch file